

```

from peft import prepare_model_for_kbit_training
import torch

bnb_config = BitsAndBytesConfig(
    load_in_4bit=USE_4BIT,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)

tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, use_fast=True)
tokenizer.padding_side = "right"
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    torch_dtype=torch.float16,
    quantization_config=bnb_config if USE_4BIT else None,
    device_map="auto"
)

model.config.use_cache = False
model.gradient_checkpointing_enable()
if USE_4BIT:
    model = prepare_model_for_kbit_training(model)

```

Step 5: Supervised fine-tuning configuration

Prepare LoRA adapter architecture and the full supervised fine-tuning configuration used by the SFTTrainer to execute parameter-efficient optimisation of the model. All core hyperparameters and training parameters have been initialised and explained in the previous steps of the pipeline. Some of the important LoRA configuration and SFT configuration parameters are described below.

```

from trl import SFTTrainer
from trl import SFTConfig
from peft import LoraConfig, TaskType
from peft import LoraConfig, get_peft_model

from peft import LoraConfig, TaskType

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_dropout=0.05,

```

```

    bias="none",
    task_type=TaskType.CAUSAL_LM,
)

sft_config = SFTConfig(
    output_dir=OUTPUT_DIR,
    num_train_epochs=EPOCHS,
    per_device_train_batch_size=BATCH_SIZE,
    gradient_accumulation_steps=GRAD_ACCUM,
    learning_rate=LR,
    lr_scheduler_type="cosine",
    warmup_ratio=WARMUP_RATIO,
    logging_steps=LOG_STEPS,
    save_steps=SAVE_STEPS,
    save_total_limit=2,

    # PURE FP16 – NO BF16 EVER
    fp16=False,
    bf16=False,
    half_precision_backend="cuda",

    # VERY IMPORTANT
    optim="paged_adamw_8bit" if USE_4BIT else "adamw_torch",
    # optim = "adamw_torch",
    max_grad_norm=1.0,
    gradient_checkpointing=True,

    # SFT
    max_length=SEQ_LEN,
    dataset_text_field="text",
    packing=False,
)

trainer = SFTTrainer(
    model=model,
    train_dataset=train_ds,
    args=sft_config,          # <- SFTConfig with all settings
    processing_class=tokenizer, # <- replaces old
    `tokenizer=` arg
    peft_config=lora_config,
)

```

r=16: Low-rank dimension of adapter matrices that controls adapter capacity versus memory.

lora_alpha=32: lora_alpha is a gain that scales the adapter's low-rank update before it is added to the frozen weight. The scaling factor is (alpha/rank). With lora_alpha=32 and r=16, the adapter contribution is effectively doubled, giving LoRA more influence over the model's behaviour while leaving the backbone untouched.

target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]: target_modules